

字库取模教程

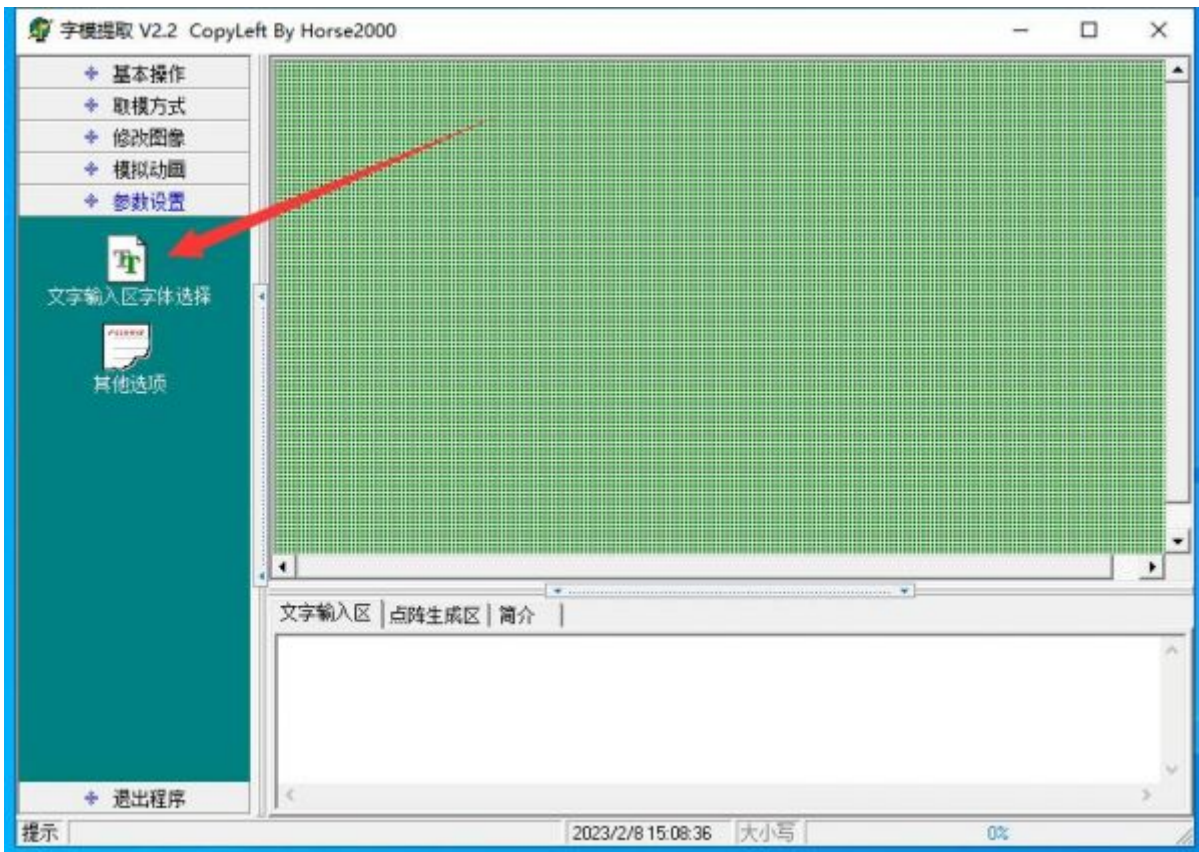
以下均以汉字为为例，并以树莓派程序与2.13 V3黑白屏幕作为讲解，其他程序/字库类似

中文字库汉字添加

这里以添加font24CN.c为例

打开字模提取软件，按照下面步骤 对软件进行配置

选择 “文字输入区字体选择”



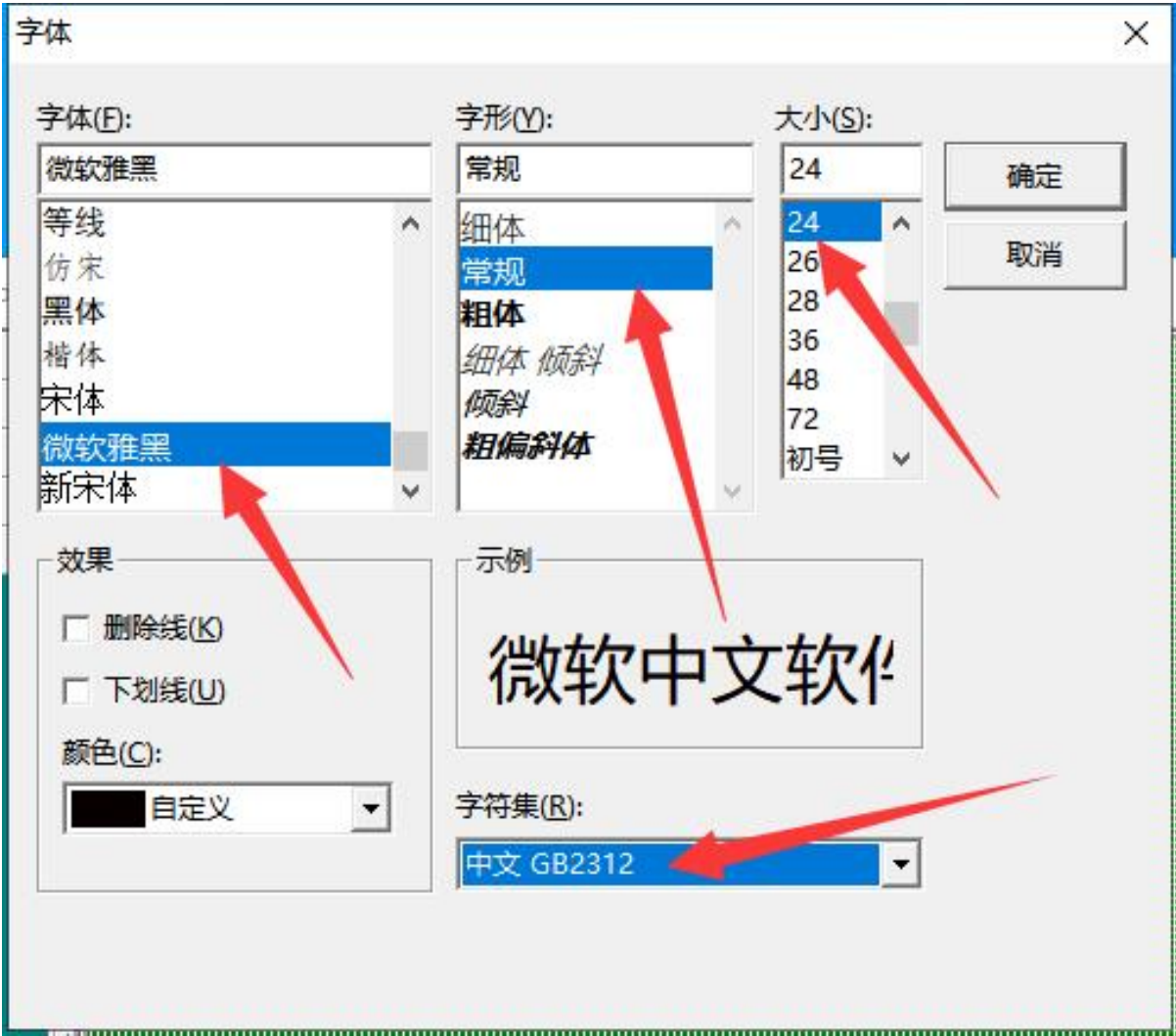
按照图片选择对应的字体型号，配置完成后点击确定

字体：微软雅黑

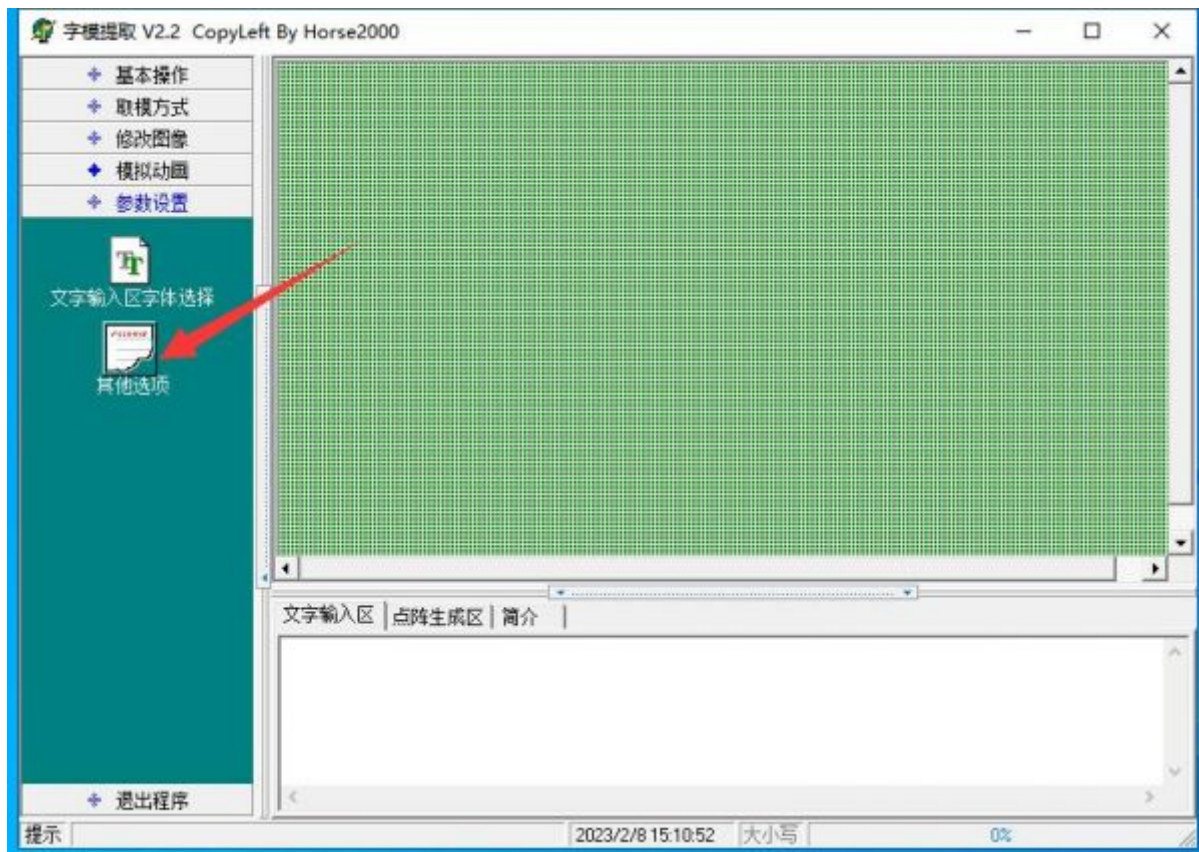
字形：常规

大小：24

字符集：GB2312



选择 “其他选项”



按照图片选择对应的配置，配置完成后点击确定

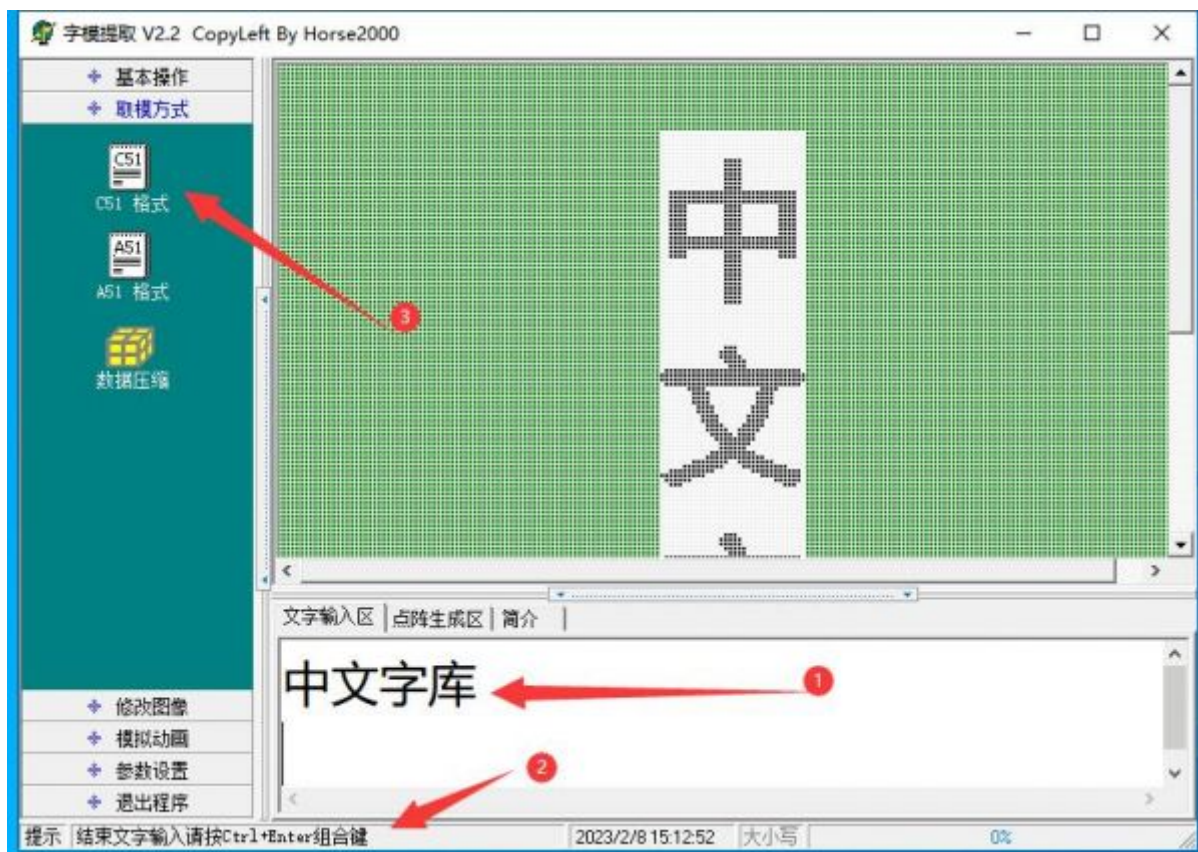
取模方式：横向取模

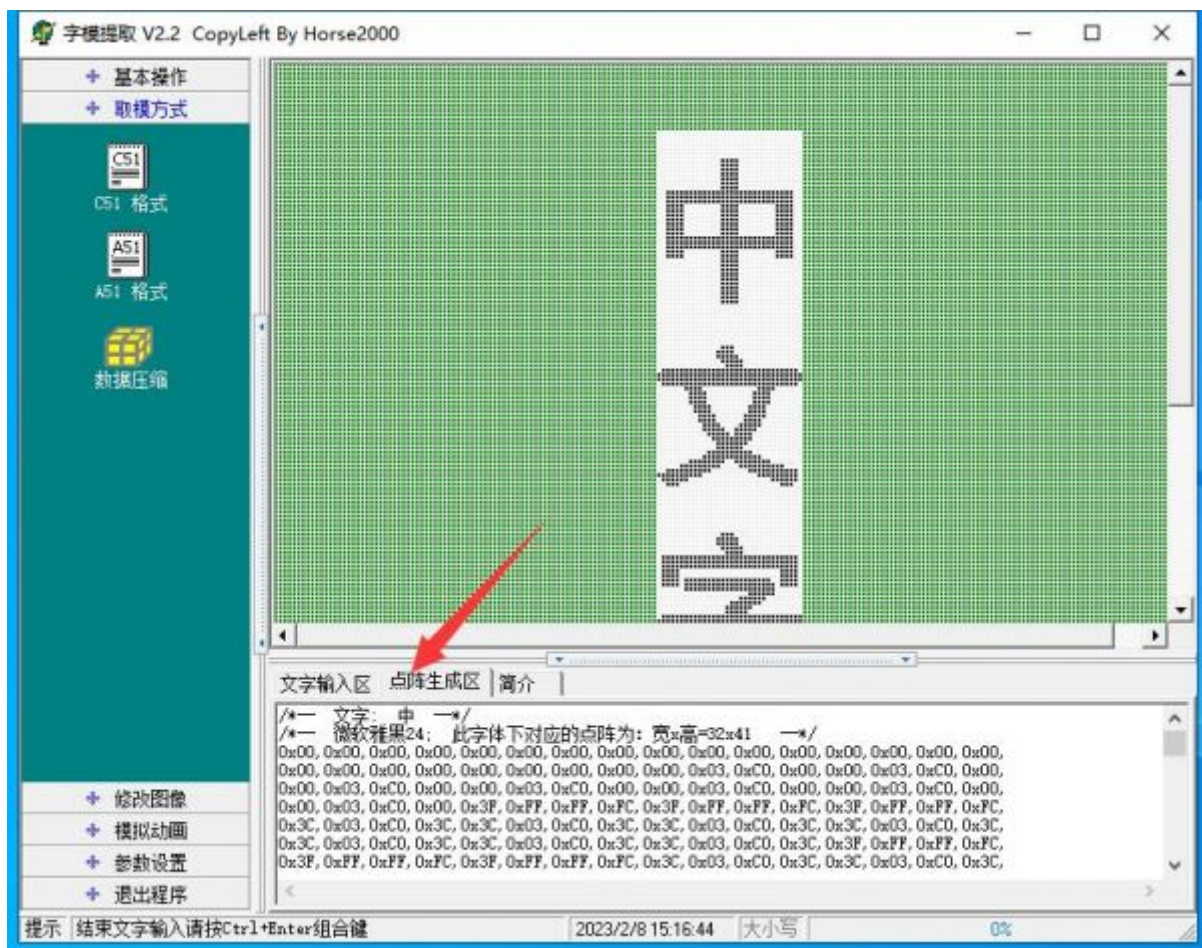
保留字节最后一位的逗号

我们后面使用C51取模，A51格式可以不用管

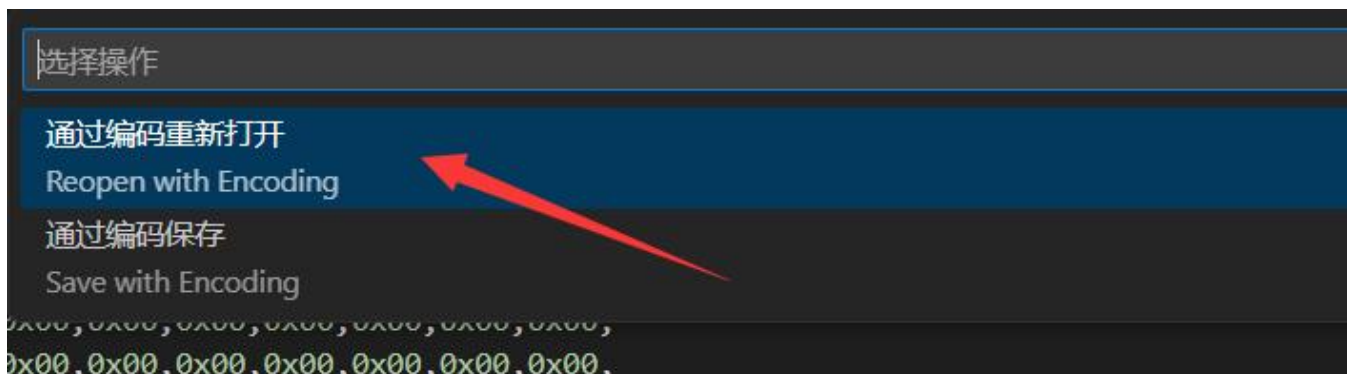


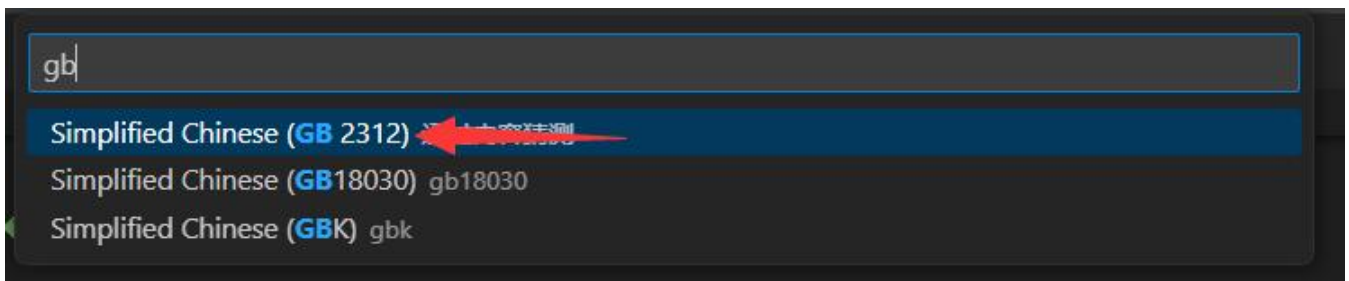
- 1、在 1 处输入要添加的汉字，输入完成后，按下 Ctrl + 回车
- 2、在 2 处就会显示输入的汉字的取模预览
- 3、点击左侧的C51格式进行取模
- 4、随后便会生成对应的十六进制数据



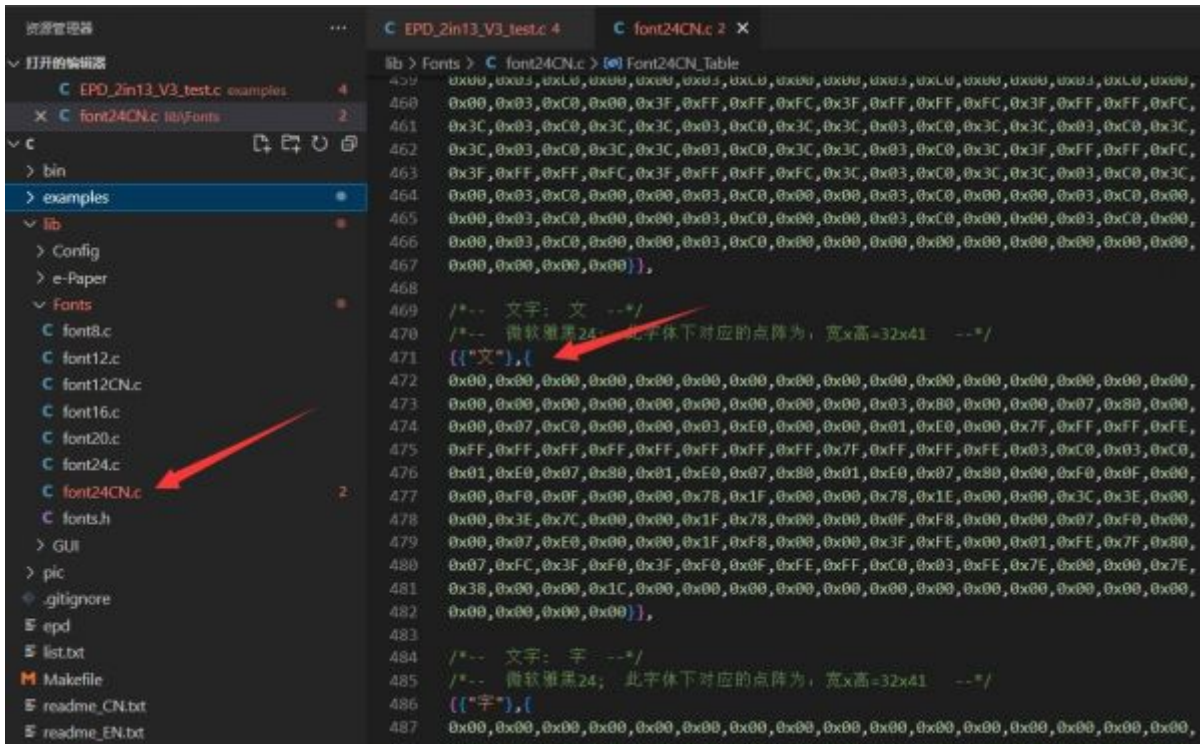


打开程序对应文件，我这里使用的软件是 VS Ccode，将 UTF-8 的格式改为 GB2312





将刚取模好的数据按照文件对应的格式，复制并修改到对应的文件中



打开文件 EPD_2in13_V3_test.c 文件，并将原先显示“微雪电子”的地方改为刚刚取模的“中文字库”，编译并运行程序，会看到原先例程显示“微雪电子”的地方已经变成了“中文字库”

源文件管理器

打开的编辑器 1 个未保存

- EPD_2in13_V3_test.c examples 4
- font24CN.c lib/Fonts 2

C

- EPD_1in54c_test.c
- EPD_1in54c_test.c
- EPD_1in54c_V2_test.c
- EPD_1in54b_test.c
- EPD_1in54b_V2_test.c
- EPD_1in54c_test.c
- EPD_1in64g_test.c
- EPD_2in7_test.c
- EPD_2in7_V2_test.c
- EPD_2in7b_test.c
- EPD_2in7b_V2_test.c
- EPD_2in9_DES_test.c
- EPD_2in9_test.c
- EPD_2in9_V2_test.c
- EPD_2in9b_V3_test.c
- EPD_2in9bc_test.c
- EPD_2in9d_test.c
- EPD_2in13_DES_test.c
- EPD_2in13_test.c
- EPD_2in13_V2_test.c
- EPD_2in13b_V3_test.c
- EPD_2in13b_V4_test.c
- EPD_2in13bc_test.c
- EPD_2in13d_test.c
- EPD_2in36g_test.c
- EPD_2in66_test.c
- EPD_2in66b_test.c
- EPD_3in0g_test.c
- EPD_3in7_test.c
- EPD_3in52_test.c
- EPD_4in01f_test.c
- EPD_4in2_test.c
- EPD_4in2_V2_test.c

examples > C EPD_2in13_V3_test.c > EPD_2in13_V3_test(void)

```
73 DEV_Delay_ms(3000);
74 #endif
75
76 #if 0 //show image for array
77     Debug("show image for array\r\n");
78     Paint_SelectImage(BlackImage);
79     Paint_Clear(WHITE);
80     Paint_DrawBMP(gImage_2in13_2);
81
82     EPD_2in13_V3_Display(BlackImage);
83     DEV_Delay_ms(2000);
84 #endif
85
86 #if 1 // Drawing on the image
87     Paint_NewImage(BlackImage, EPD_2in13_V3_WIDTH, EPD_2in13_V3_HEIGHT, 90, WHITE);
88     Debug("Drawing\r\n");
89     //1.Select image
90     Paint_SelectImage(BlackImage);
91     Paint_Clear(WHITE);
92
93     // 2.Drawing on the image
94     Paint_DrawPoint(5, 10, BLACK, DOT_PIXEL_1X1, DOT_STYLE_DFT);
95     Paint_DrawPoint(5, 25, BLACK, DOT_PIXEL_2X2, DOT_STYLE_DFT);
96     Paint_DrawPoint(5, 40, BLACK, DOT_PIXEL_3X3, DOT_STYLE_DFT);
97     Paint_DrawPoint(5, 55, BLACK, DOT_PIXEL_4X4, DOT_STYLE_DFT);
98
99     Paint_DrawLine(20, 10, 70, 60, BLACK, DOT_PIXEL_1X1, LINE_STYLE_SOLID);
100    Paint_DrawLine(70, 10, 20, 60, BLACK, DOT_PIXEL_1X1, LINE_STYLE_SOLID);
101    Paint_DrawRectangle(20, 10, 70, 60, BLACK, DOT_PIXEL_1X1, DRAW_FILL_EMPTY);
102    Paint_DrawRectangle(85, 10, 135, 60, BLACK, DOT_PIXEL_1X1, DRAW_FILL_FULL);
103
104    Paint_DrawLine(45, 15, 45, 55, BLACK, DOT_PIXEL_1X1, LINE_STYLE_DOTTED);
105    Paint_DrawLine(25, 35, 70, 35, BLACK, DOT_PIXEL_1X1, LINE_STYLE_DOTTED);
106    Paint_DrawCircle(45, 35, 20, BLACK, DOT_PIXEL_1X1, DRAW_FILL_EMPTY);
107    Paint_DrawCircle(110, 35, 20, WHITE, DOT_PIXEL_1X1, DRAW_FILL_FULL);
108
109    Paint_DrawString_EN(140, 15, "waveshare", &Font16, BLACK, WHITE);
110    Paint_DrawNum(140, 40, 123456789, &Font16, BLACK, WHITE);
111
112    Paint_DrawString_CN(140, 60, "你好abc", &Font12CN, BLACK, WHITE);
113    Paint_DrawString_CN(5, 65, "创客电子", &Font24CN, WHITE, BLACK);
114
115    EPD_2in13_V3_Display_Base(BlackImage);
116    DEV_Delay_ms(3000);
```

源文件管理器

打开的编辑器

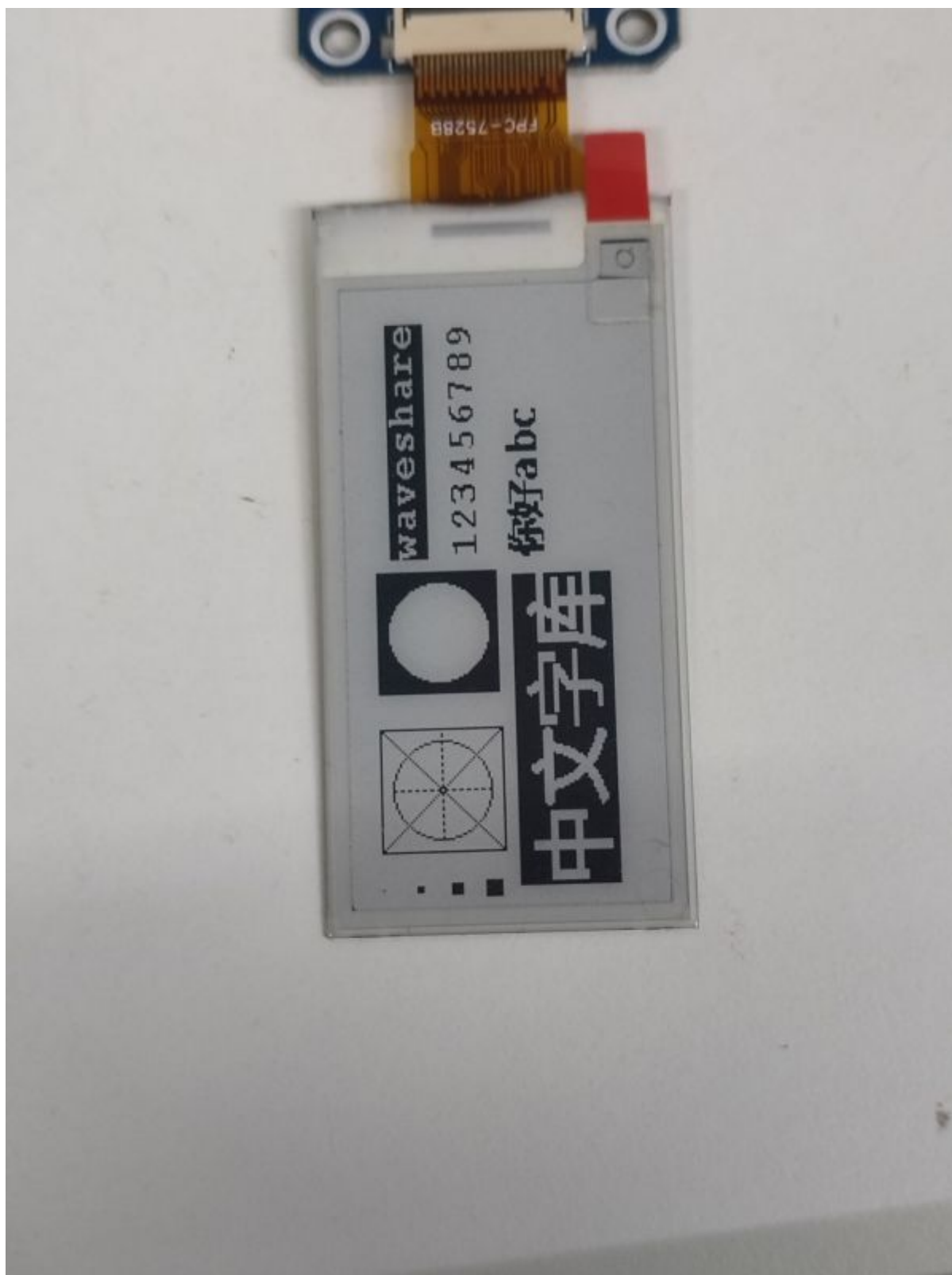
- X C EPD_2in13_V3_test.c examples 4
- C font24CN.c lib/Fonts 2

C

- > bin
- > examples
- EPD_1in02d_test.c
- EPD_1in54c_DES_test.c
- EPD_1in54c_test.c
- EPD_1in54c_V2_test.c
- EPD_1in54b_test.c
- EPD_1in54b_V2_test.c
- EPD_1in54c_test.c
- EPD_1in64g_test.c
- EPD_2in7_test.c
- EPD_2in7_V2_test.c
- EPD_2in7b_test.c
- EPD_2in7b_V2_test.c
- EPD_2in9_DES_test.c
- EPD_2in9_test.c
- EPD_2in9_V2_test.c
- EPD_2in9b_V3_test.c
- EPD_2in9bc_test.c
- EPD_2in9d_test.c
- EPD_2in13_DES_test.c
- EPD_2in13_test.c
- EPD_2in13_V2_test.c
- EPD_2in13_V3_test.c 4
- EPD_2in13b_V3_test.c
- EPD_2in13b_V4_test.c
- EPD_2in13bc_test.c
- EPD_2in13d_test.c
- EPD_2in36g_test.c
- EPD_2in66_test.c
- EPD_2in66b_test.c
- EPD_3in0g_test.c
- EPD_3in7_test.c
- EPD_3in52_test.c
- EPD_4in01f_test.c

examples > C EPD_2in13_V3_test.c > EPD_2in13_V3_test(void)

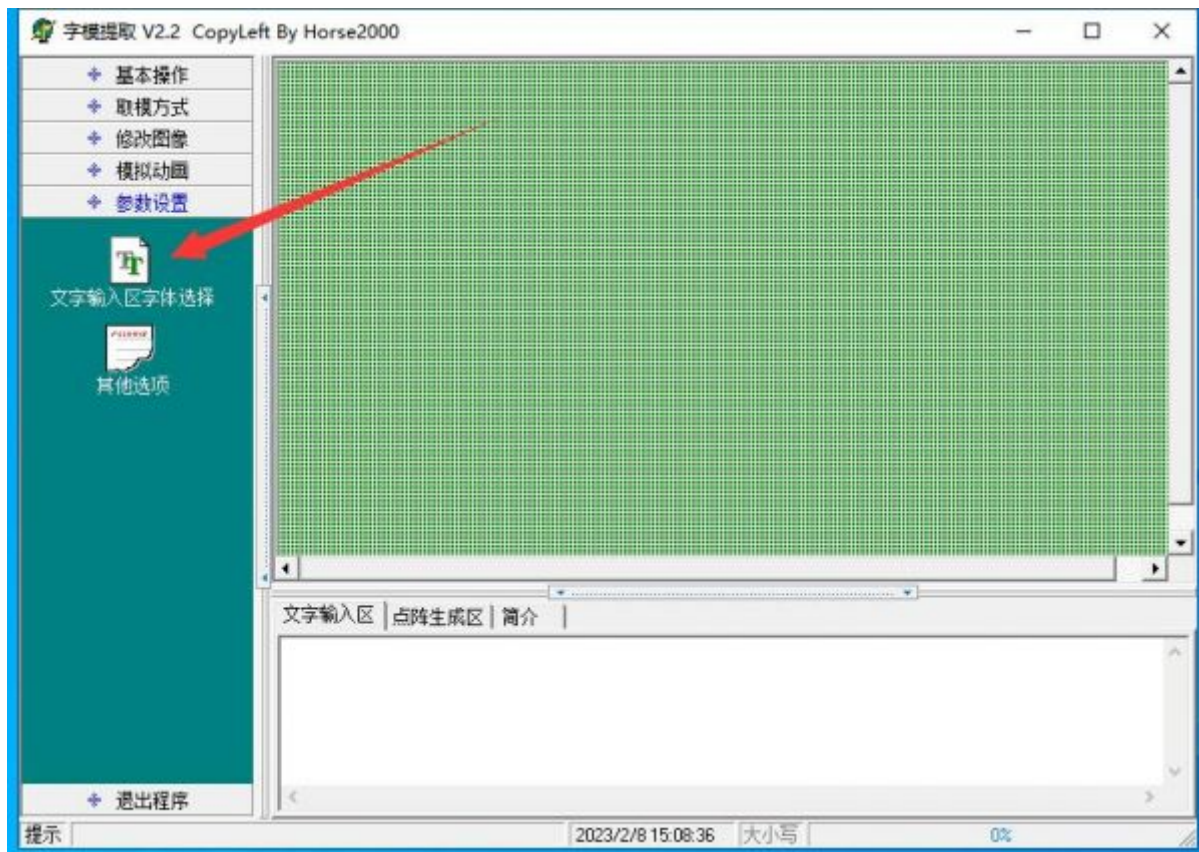
```
73 DEV_Delay_ms(3000);
74 #endif
75
76 #if 0 //show image for array
77     Debug("show image for array\r\n");
78     Paint_SelectImage(BlackImage);
79     Paint_Clear(WHITE);
80     Paint_DrawBMP(gImage_2in13_2);
81
82     EPD_2in13_V3_Display(BlackImage);
83     DEV_Delay_ms(2000);
84 #endif
85
86 #if 1 // Drawing on the image
87     Paint_NewImage(BlackImage, EPD_2in13_V3_WIDTH, EPD_2in13_V3_HEIGHT, 90, WHITE);
88     Debug("Drawing\r\n");
89     //1.Select image
90     Paint_SelectImage(BlackImage);
91     Paint_Clear(WHITE);
92
93     // 2.Drawing on the image
94     Paint_DrawPoint(5, 10, BLACK, DOT_PIXEL_1X1, DOT_STYLE_DFT);
95     Paint_DrawPoint(5, 25, BLACK, DOT_PIXEL_2X2, DOT_STYLE_DFT);
96     Paint_DrawPoint(5, 40, BLACK, DOT_PIXEL_3X3, DOT_STYLE_DFT);
97     Paint_DrawPoint(5, 55, BLACK, DOT_PIXEL_4X4, DOT_STYLE_DFT);
98
99     Paint_DrawLine(20, 10, 70, 60, BLACK, DOT_PIXEL_1X1, LINE_STYLE_SOLID);
100    Paint_DrawLine(70, 10, 20, 60, BLACK, DOT_PIXEL_1X1, LINE_STYLE_SOLID);
101    Paint_DrawRectangle(20, 10, 70, 60, BLACK, DOT_PIXEL_1X1, DRAW_FILL_EMPTY);
102    Paint_DrawRectangle(85, 10, 135, 60, BLACK, DOT_PIXEL_1X1, DRAW_FILL_FULL);
103
104    Paint_DrawLine(45, 15, 45, 55, BLACK, DOT_PIXEL_1X1, LINE_STYLE_DOTTED);
105    Paint_DrawLine(25, 35, 70, 35, BLACK, DOT_PIXEL_1X1, LINE_STYLE_DOTTED);
106    Paint_DrawCircle(45, 35, 20, BLACK, DOT_PIXEL_1X1, DRAW_FILL_EMPTY);
107    Paint_DrawCircle(110, 35, 20, WHITE, DOT_PIXEL_1X1, DRAW_FILL_FULL);
108
109    Paint_DrawString_EN(140, 15, "waveshare", &Font16, BLACK, WHITE);
110    Paint_DrawNum(140, 40, 123456789, &Font16, BLACK, WHITE);
111
112    Paint_DrawString_CN(140, 60, "你好abc", &Font12CN, BLACK, WHITE);
113    Paint_DrawString_CN(5, 65, "中文序", &Font24CN, WHITE, BLACK);
114
115    EPD_2in13_V3_Display_Base(BlackImage);
116    DEV_Delay_ms(3000);
```



新字库添加

我们这里添加 font48CN.c 文件 打开字模提取软件，按照下面图片对应软件进行配置

选择 “文字输入区字体选择”



按照图片选择对应的字体型号，配置完成后点击确定

字体：微软雅黑

字形：常规

大小：48

字符集：GB2312

字体



字体(F):

微软雅黑

等线

仿宋

黑体

楷体

宋体

微软雅黑

新宋体

字形(Y):

常规

细体

常规

粗体

细体 倾斜

倾斜

粗偏斜体

大小(S):

48

48

72

初号

小初

一号

小一

二号

确定

取消

效果

☐ 删除线(K)

☐ 下划线(U)

颜色(C):

自定义

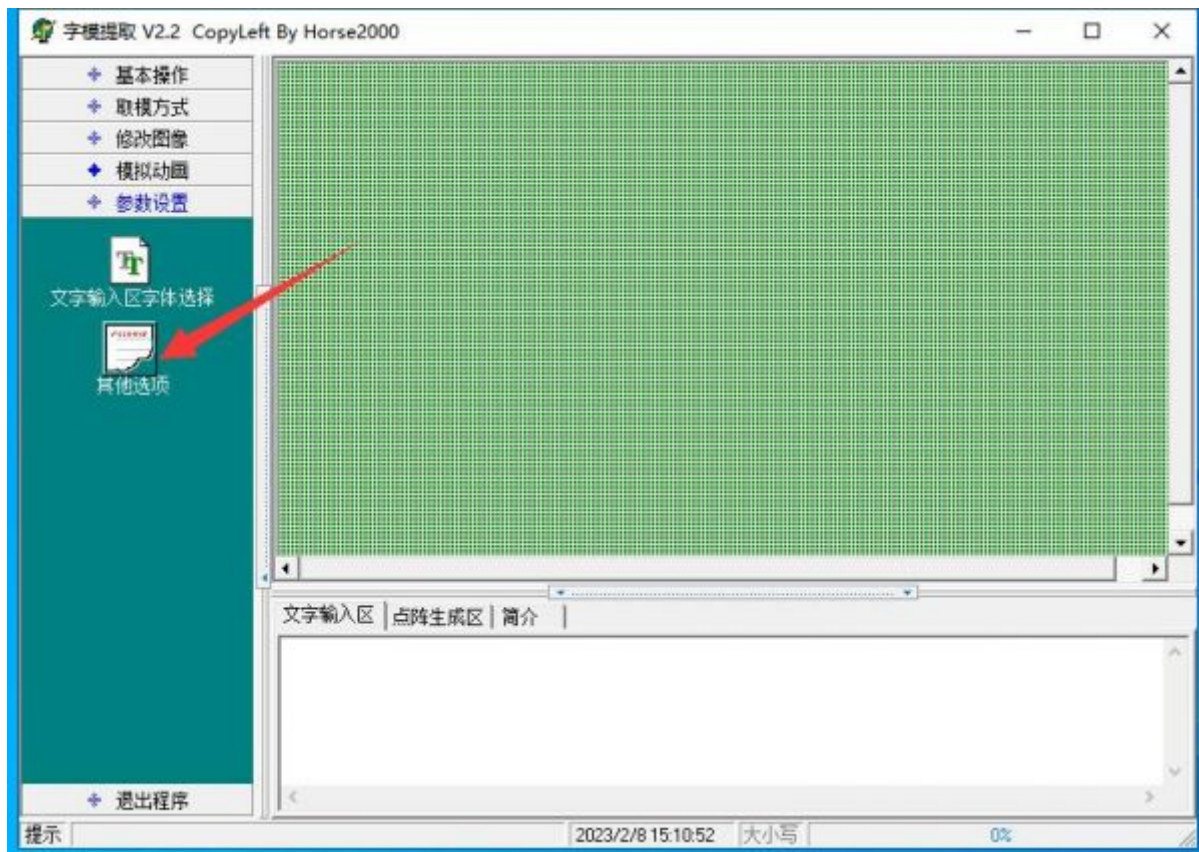
示例

微软中

字符集(R):

中文 GB2312

选择 “其他选项”



按照图片选择对应的配置，配置完成后点击确定

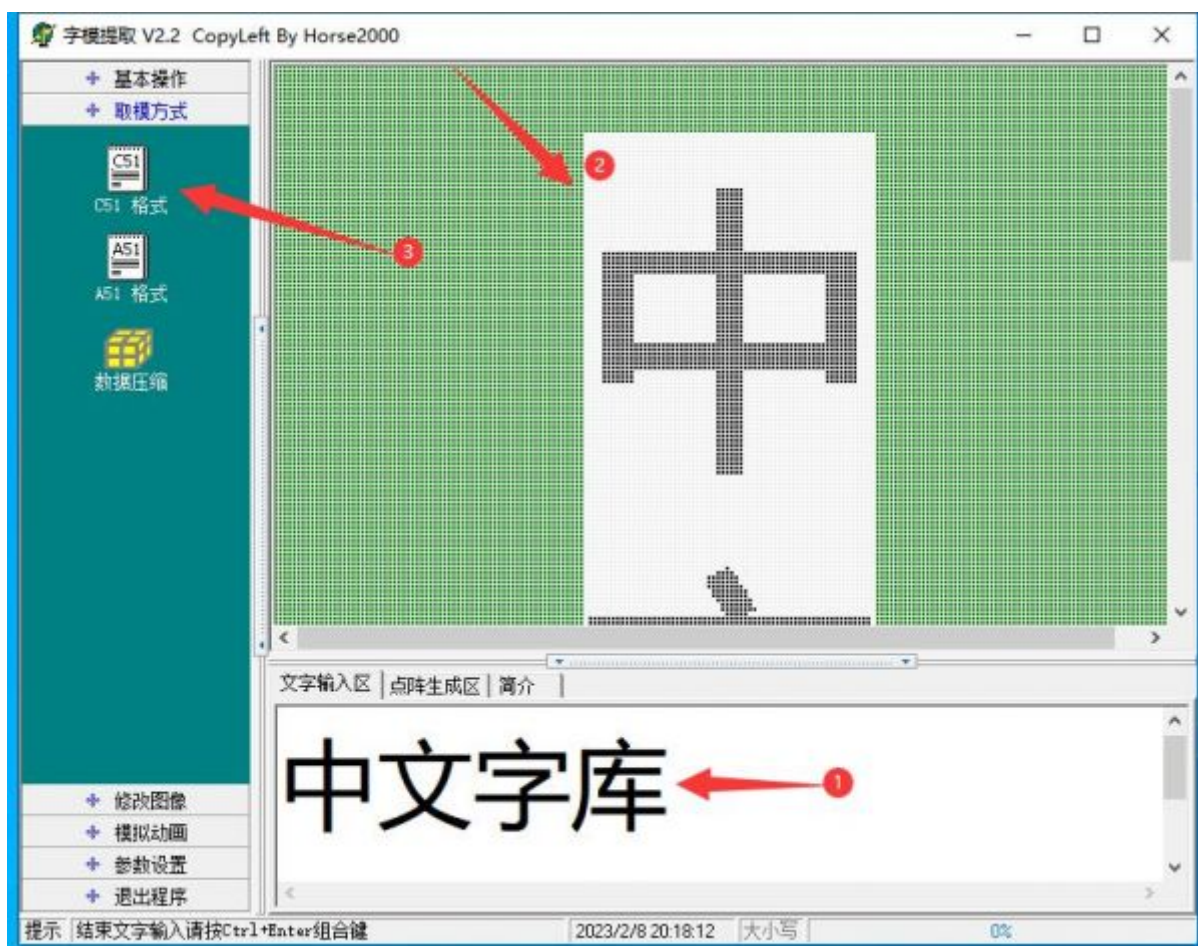
取模方式：横向取模

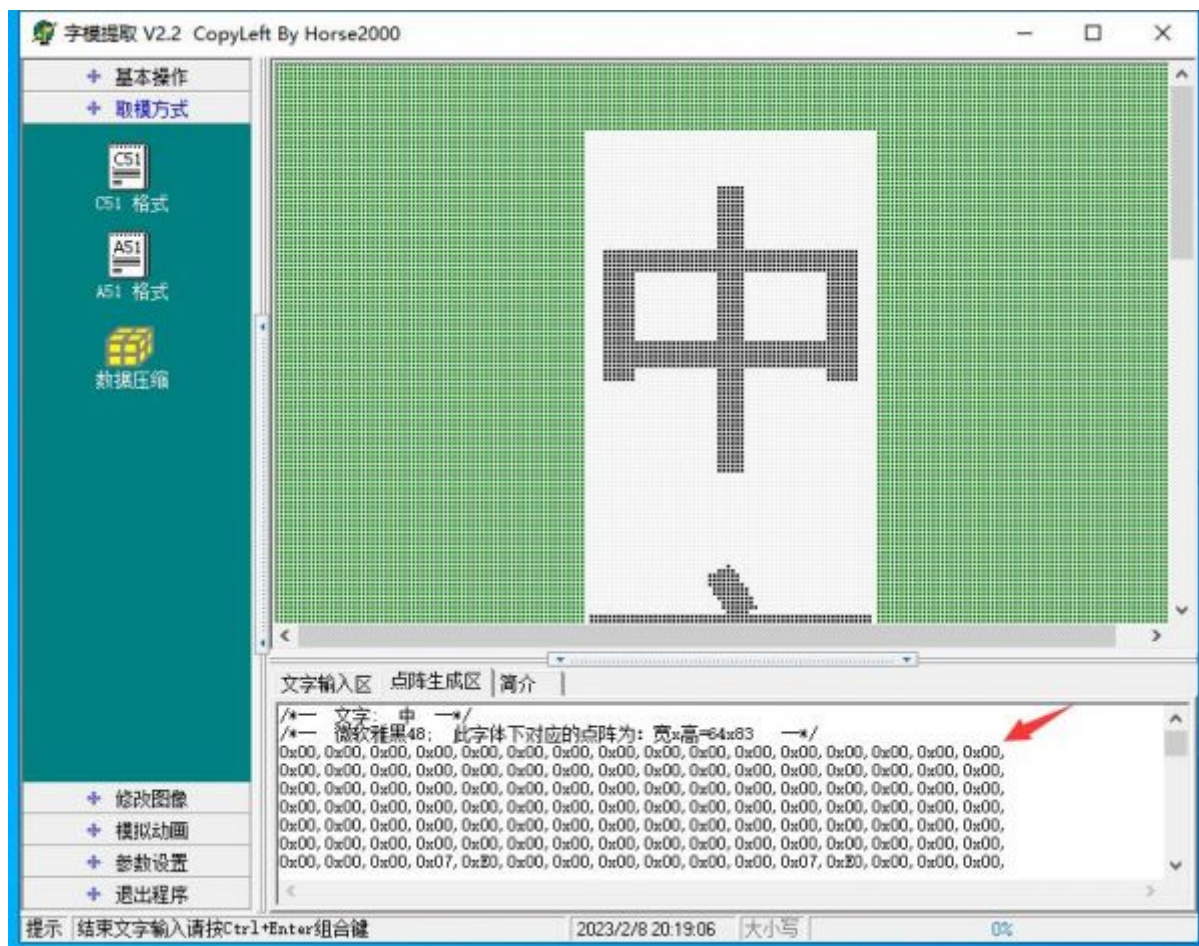
保留字节最后一位的逗号

我们后面使用C51取模，A51格式可以不用管



- 1、在 1 处输入要添加的汉字，输入完成后，按下 Ctrl + 回车
- 2、在 2 处就会显示输入的汉字的取模预览
- 3、点击左侧的C51格式进行取模
- 4、随后便会生成对应的十六进制数据





在对应文件目录下新建 font48CN.c 文件

名称	修改日期	类型	大小
font8.c	2021/11/1 14:37	C 文件	17 KB
font12.c	2021/11/1 14:37	C 文件	26 KB
font12CN.c	2021/11/1 14:37	C 文件	6 KB
font16.c	2021/11/1 14:37	C 文件	47 KB
font20.c	2021/11/1 14:37	C 文件	63 KB
font24.c	2021/11/1 14:37	C 文件	95 KB
font24CN.c	2023/1/12 15:15	C 文件	31 KB
font48CN.c	2023/1/12 15:39	C 文件	16 KB
fonts.h	2023/1/12 15:44	H 文件	4 KB

[illegible]

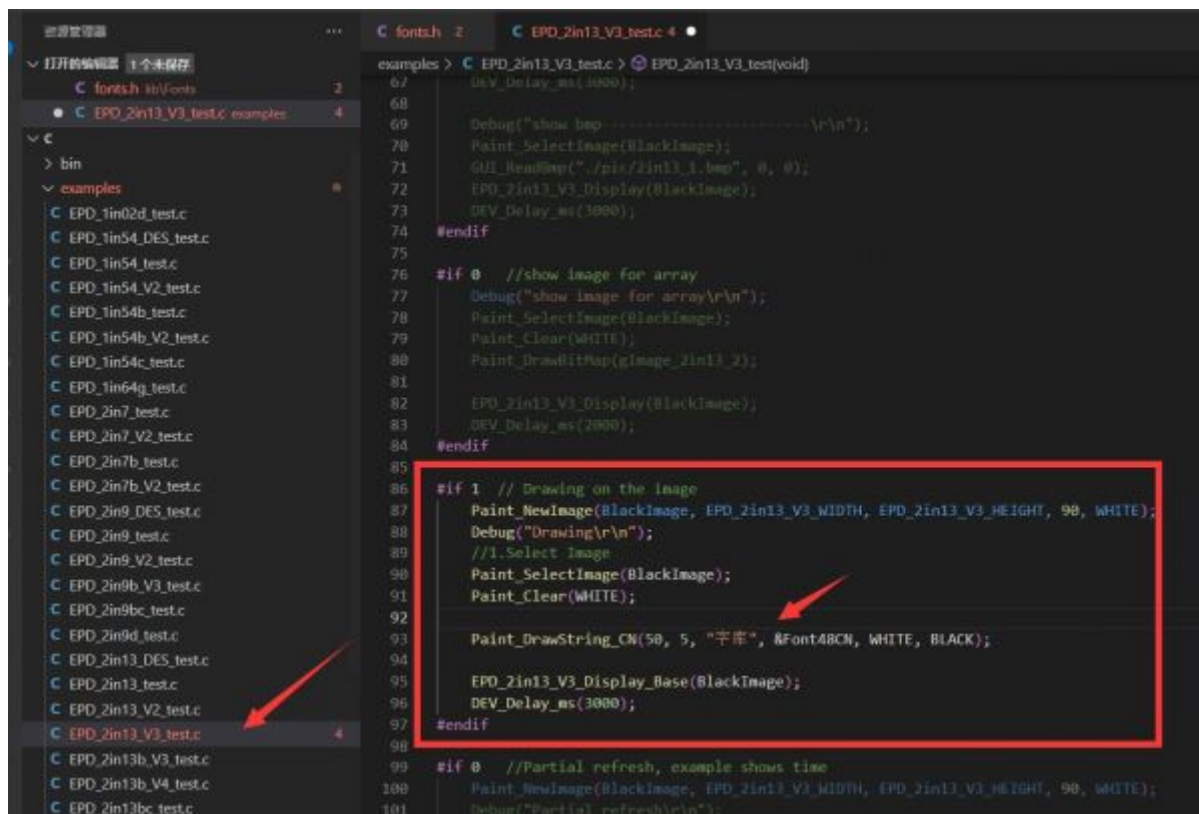
The screenshot shows the Visual Studio Code interface. On the left, the 'File Explorer' pane displays the project structure: 'lib' > 'e-Paper' > 'Fonts'. The 'Fonts' folder is expanded, showing files like 'font8.c', 'font12.c', 'font12CN.c', 'font16.c', 'font20.c', 'font24.c', 'font24CN.c', 'font48CN.c', and 'fonts.h'. The 'fonts.h' file is selected. On the right, the 'Source Code' pane shows the content of 'fonts.h'. The code includes a comment in Chinese: '/*最大字体为，微软雅黑 48号 (64x83) */'. Below this, two macros are defined: '#define MAX_HEIGHT_FONT 83' and '#define MAX_WIDTH_FONT 64'. Red arrows point from these definitions to the file explorer, indicating the source of the dimensions used in the previous image.


```

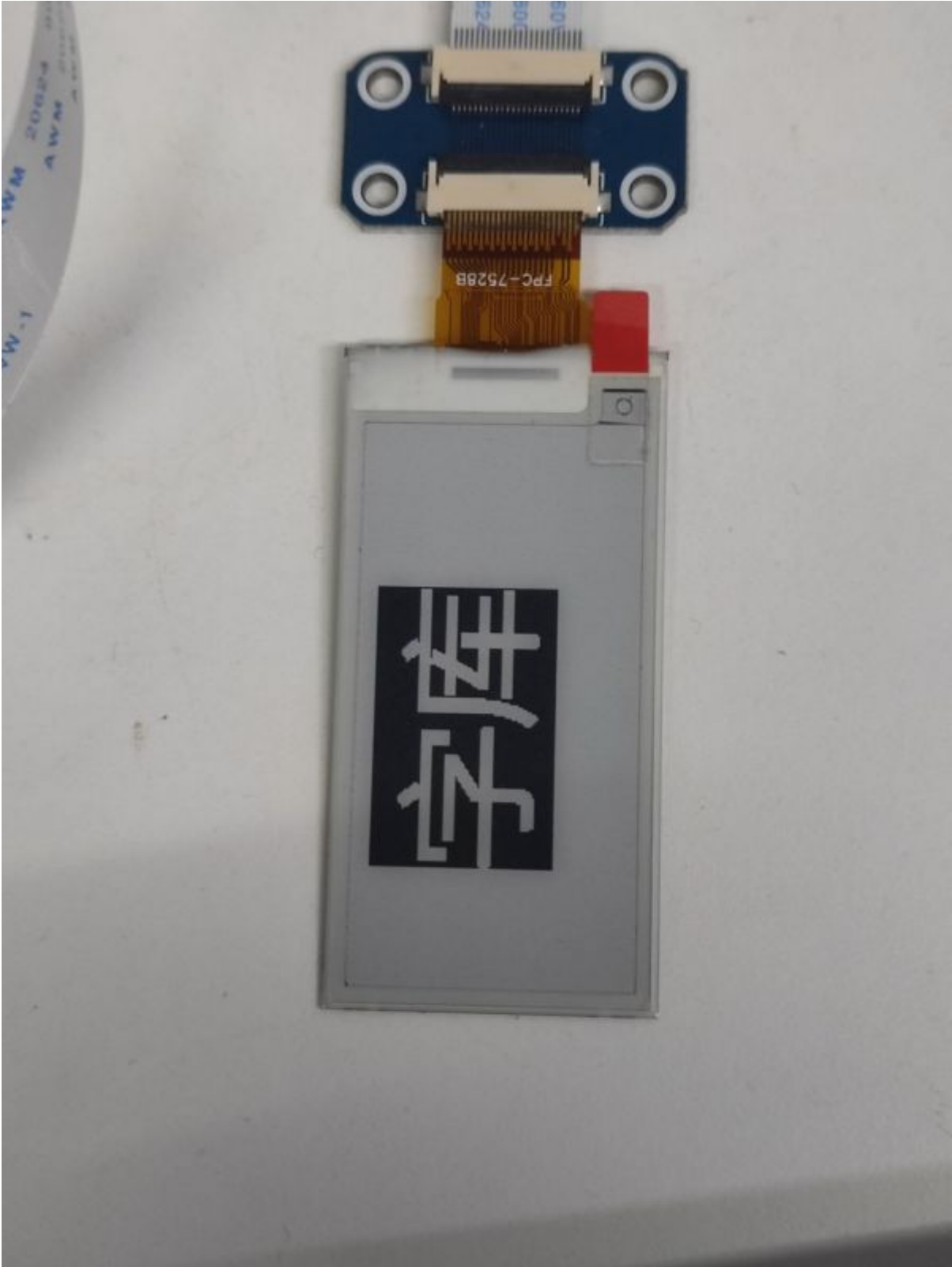
81
82  extern sFONT Font24;
83  extern sFONT Font20;
84  extern sFONT Font16;
85  extern sFONT Font12;
86  extern sFONT Font8;
87
88  extern cFONT Font12CN;
89  extern cFONT Font24CN;
90  extern cFONT Font48CN;
91  #ifdef __cplusplus
92  }
93  #endif
94
95  #endif /* __FONT_H */
96
97

```

打开文件 EPD_2in13_V3_test.c 文件，修改成图示的程序



显示效果如下

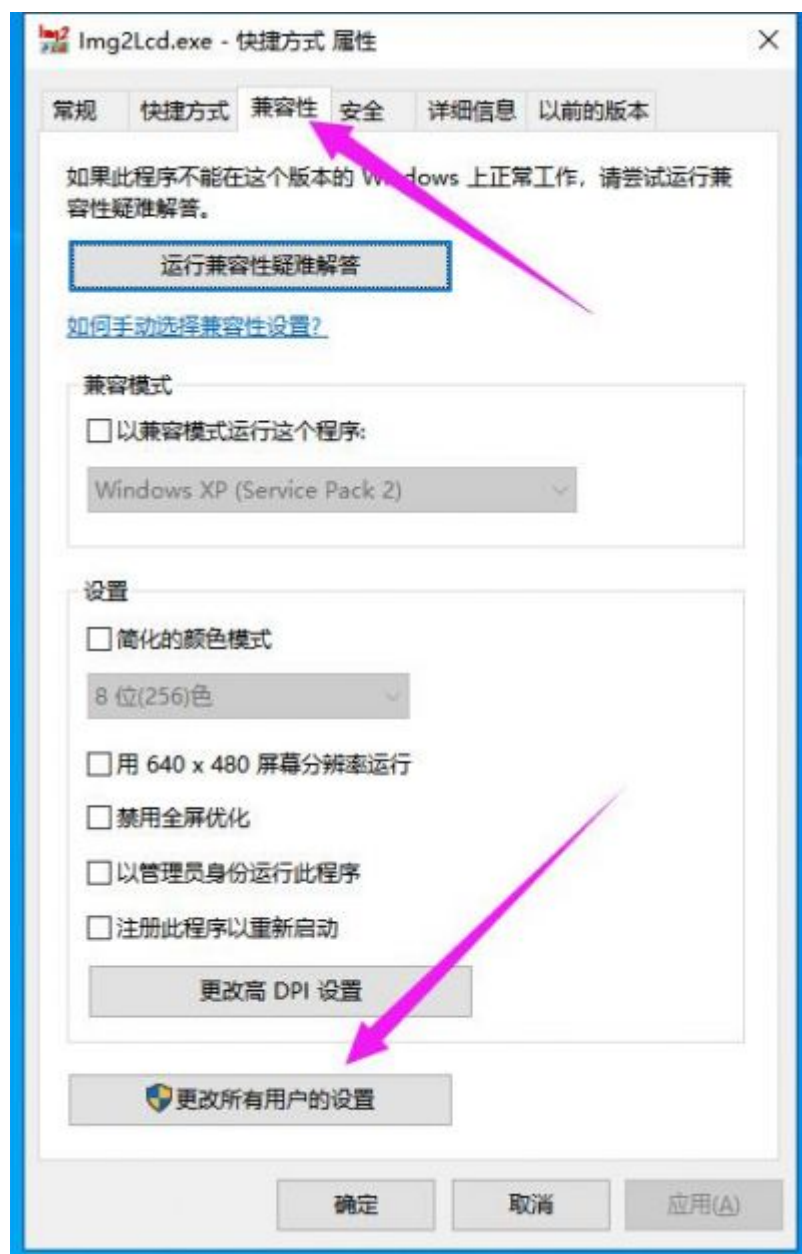


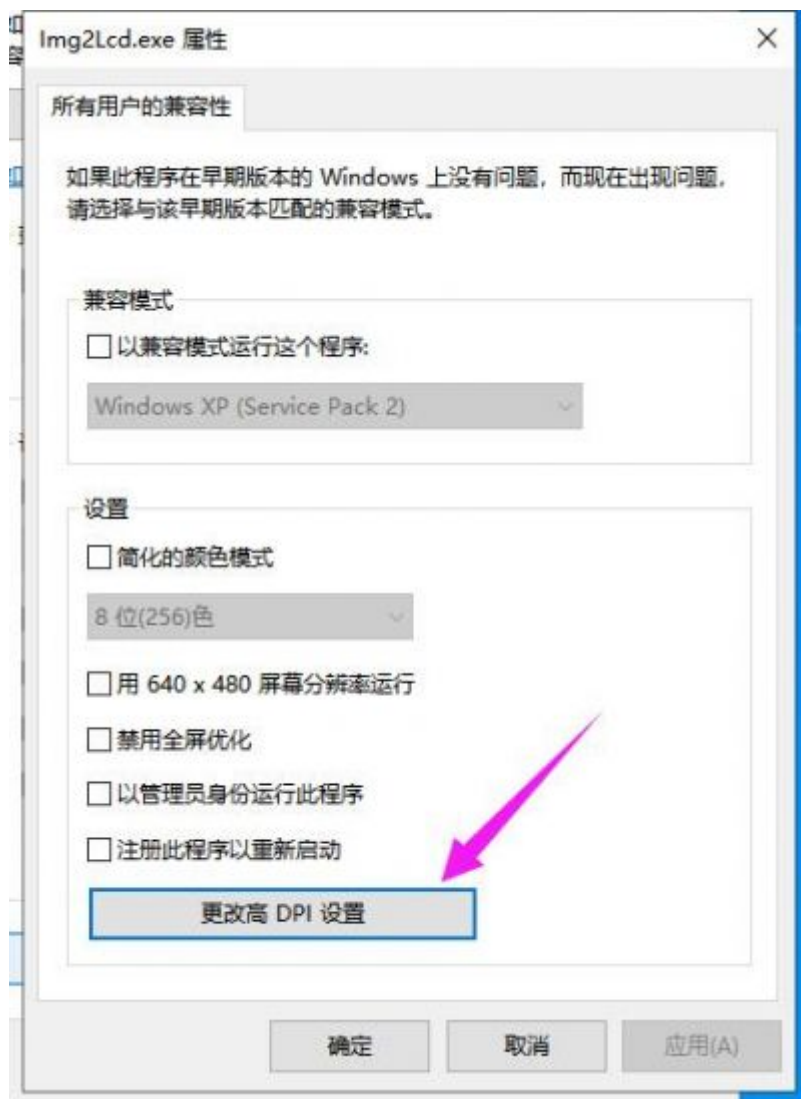
FAQ

问题：按照我们的配置，取出的字体大小和我们的不一样

问题原因：电脑系统兼容性的原因

解决办法：设置好对应的兼容性即可，右击软件、选择属性





Img2Lcd.exe 属性



为此程序选择高 DPI 设置。

程序 DPI

☐ 使用此设置修复此程序的缩放问题，而不是“设置”中的缩放问题

[打开高级缩放设置](#)

如果在登录到 Windows 之后主显示器的 DPI 发生更改，则程序可能看起来较模糊。Windows 可以根据你在打开此程序时为主显示器设置的 DPI，尝试修复该程序的缩放问题。

执行以下操作时使用为我的主显示器设置的 DPI

已登录到 Windows

[了解详细信息](#)

高 DPI 缩放替代

☐ 替代高 DPI 缩放行为。

缩放执行:

应用程序

确定

取消